

Indian Institute of Technology Dharwad

Data Structures and Algorithms: Version A

Quiz 1 – February 2, 2026

Roll Number: _____

Duration: 45 minutes

Max marks: 30

General Instructions:

1. In questions involving code snippets with blanks, each blank corresponds to **exactly one valid statement**. Writing multiple statements in a single blank will not be evaluated.
2. Answers are marked either **1** or **0**. There is **no partial marking** (e.g., no 0.5 marks).
3. The **last blank sheet** of the answer booklet may be used as a rough sheet. **No additional sheets** will be provided.
4. Use of mobile phones or any other **smart electronic devices** is strictly prohibited. If a student is found in possession of such devices during the examination, it will lead to **immediate expulsion from the course**.

1. Determine the output of the program.

```
#include <stdio.h>
int main() {
    int a = 5, b = 10;
    int *p = &a, *q = &b;
    *p = *q;
    q = p;
    *q = 20;
    printf("%d %d", a, b);
    return 0;
}
```

~~20~~ ~~20~~ 20 10

2. Consider the following C program:

```
void main() {
    int arr[3] = {1, 2, 3};
    int *p = arr;
    printf("%d ", *(p + 1));
    p++;
    arr++;
    printf("%d", *p);
}
```

Which of the following is **correct**?

- (51)
- (a) The program compiles and prints 2 2
 - (b) The program compiles and prints 2 3
 - ☒ (c) The program fails to compile
 - (d) The program results in undefined behaviour at runtime
3. The output of the below program is

```
void modify(int **p, int *q) {
    *p = q;
    **p = **p + 5;
}
```

```
int main() {
    int x = 3, y = 7;
    int *r = &x;
    modify(&r, &y);
}
```

```
printf("%c %d", x, y);
return 0;
}
```

3 12

4. What is the effect of calling fun(head)?

```
void fun(struct node *p) {
    while (p->next != NULL) {
        p = p->next;
    }
    p->next = head;
}
```

segmentation fault

- ☒ (a) The linked list becomes circular
 - (b) The last node of the list is deleted
 - (c) A duplicate of the first node is added at the end
 - ☒ (d) The program results in a segmentation fault
5. Which of the following statements is **TRUE** for the below function?

```
void func(struct node *p) {
    if (p == NULL || p->next == NULL)
        return;
    p->data = p->next->data;
    p->next = p->next->next;
}
```

- (a) The function correctly deletes any node, including the last node
- ☒ (b) The function deletes the node pointed to by p without using the head pointer
- (c) The function causes memory leakage only when p points to the first node
- (d) The function swaps data between two nodes but does not delete any node

6. Fill in the blanks so that the function correctly returns the middle node of the list.

```

Node* findMiddle(Node *head) {
    Node *slow = head;
    Node *fast = head;
    if (head == NULL)
        return NULL;

    while (fast != NULL (i.e. fast->next != NULL)) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

```

7. What does the below function do?

```

Node* func(Node *head, int n) {
    Node *p = head;
    Node *q = head;
    while (n-- > 0) {
        if (p == NULL)
            return NULL;
        p = p->next;
    }
    while (p != NULL) {
        p = p->next;
        q = q->next;
    }
    return q;
}

```

- (a) Returns the address of the $(n + 1)^{\text{th}}$ node from the end of the list
- (b) Returns the address of the $(n + 1)^{\text{th}}$ node from the beginning of the list
- (c) Returns the address of the $(n - 1)^{\text{th}}$ node from the end of the list
- (d) Returns the address of the $(n - 1)^{\text{th}}$ node from the beginning of the list
- (e) Returns the address of the n^{th} node from the beginning of the list
- ☒ (f) Returns the address of the n^{th} node from the end of the list

8. What does the below function do?

```

Node* func(Node *head, int k) {
    if (head == NULL || k == 0)
        return head;
    Node *p = head;
    int count = 1;
    while (count < k && p != NULL) {
        p = p->next;
        count++;
    }
    if (p == NULL) return head;

    Node *q = p;
    while (q->next != NULL)
        q = q->next;

    q->next = head;
}

```

```

    head = p->next;
    p->next = NULL;
    return head;
}

```

- (a) Rotates the singly linked list to the right by k positions
- ☒ (b) Rotates the singly linked list to the left by k positions
- (c) Reverses the singly linked list in groups of size k
- (d) Moves the last k nodes of the list to the front

9. What does the below function do?

```

Node* func(Node *head) {
    Node *prev = NULL;
    Node *curr = head;
    Node *next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

```

- (a) Rotates the singly linked list by one position
- (b) Deletes alternate nodes of the linked list
- (c) Finds the middle node of the linked list
- ☒ (d) Reverses the singly linked list

10. What does the below function do?

```

Node* func(Node *head, int x, int y) {
    if (x == y)
        return head;

    Node *prevX = NULL, *currX = head;
    while (currX != NULL && currX->data != x) {
        prevX = currX;
        currX = currX->next;
    }

    Node *prevY = NULL, *currY = head;
    while (currY != NULL && currY->data != y) {
        prevY = currY;
        currY = currY->next;
    }

    if (currX == NULL || currY == NULL)
        return head;

    if (prevX != NULL)
        prevX->next = currY;
    else
        head = currY;

    if (prevY != NULL)
        prevY->next = currX;
    else
        head = currX;
}

```

```

Node *temp = currX->next;
currX->next = currY->next;
currY->next = temp;

return head;
}

```

- Swaps the data values of the nodes containing x and y
- Reverses the linked list between the nodes containing x and y
- Swaps the positions of the nodes containing x and y by modifying links
- Deletes the nodes containing x and y from the list

11. Consider the following function. Given that the doubly linked list is sorted in ascending order, what does the below function do?

```

Node* func(Node *head, int data) {
    Node *newNode = createNode(data);

    if (head == NULL) return newNode;

    if (data <= head->data) {
        newNode->next = head;
        head->prev = newNode;
        return newNode;
    }

    Node *curr = head;
    while (curr->next != NULL
        && curr->next->data < data)
        curr = curr->next;

    newNode->next = curr->next;
    if (curr->next != NULL)
        curr->next->prev = newNode;

    curr->next = newNode;
    newNode->prev = curr;

    return head;
}

```

- Inserts the new node immediately before the first node whose value is greater than or equal to data, but does not correctly update backward links
- Inserts the new node after the last node whose value is less than data, but may violate the sorted order when duplicate values exist
- Inserts a new node into the sorted doubly linked list while maintaining the sorted order
- Inserts the new node at a position determined only by forward traversal, ignoring the prev pointers

12. Complete the following function for deleting a node from a doubly linked list using a **single pointer p**. No other pointer must be used.

```

void deleteNode(Node *p) {
    if (p == NULL) return;

    if (p->prev != NULL)
        p->prev->next = p->next;

    if (p->next != NULL)
        p->next->prev = p->prev;

    free(p);
}

```

13. Complete the following function that checks whether a doubly linked list is a palindrome:

```

int isPalindrome(Node *head) {
    if (head == NULL)
        return 1;

    Node *left = head;
    Node *right = head;

    while (right->next != NULL)
        right = right->next;

    while (left != right && right->next != left) {
        if (left->data != right->data)
            return 0;

        left = left->next;
        right = right->prev;
    }

    return 1;
}

```

14. Fill in the blanks so that the function correctly inserts the new node at the rear of the double-ended circular queue.

Constraint: Only the head pointer must be used. No temporary pointer variables are allowed.

```

Node* insertRear(Node *head, int data) {
    Node *newNode = createNode(data);
    if (head == NULL) return newNode;

    newNode->next = head;
    newNode->prev = head->prev;
    head->prev->next = newNode;
    head->prev = newNode;
    return head;
}

```

15. Consider the following function. Assume that the array represents a priority queue, where a smaller value indicates a higher priority.


```

int func(int arr[], int *n) {
    int idx = 0;

    for (int i = 1; i < *n; i++) {
        if (arr[i] < arr[idx])
            idx = i;
    }

    int item = arr[idx];

    for (int i = idx; i < *n - 1; i++)
        arr[i] = arr[i + 1];

    (*n)--;

    return item;
}

```

What does the above function do?

- (a) Deletes and returns the element inserted first into the queue
- ☒ (b) Deletes and returns the element with the highest priority from the queue
- (c) Deletes and returns the element with the lowest priority from the queue
- (d) Deletes all elements from the queue

16. Assume a double ended queue (deque) is implemented using a circular array of size N . The indices `first` and `last` represent the front and rear of the queue respectively.

(a) **Insert at Front:**

Fill in the blank to correctly update the index when inserting an element at the front of a non-empty deque.

$$\text{first} = (\text{first} - 1 + N) \% N;$$

(b) **Delete from Rear:**

Fill in the blank to correctly update the index when deleting an element from the rear of a non-empty deque.

$$\text{last} = (\text{last} - 1 + N) \% N;$$

17. In a memory efficient doubly linked list, each node stores the XOR of the addresses of the previous and next nodes in a field named `npv`.

Assume `prev` is a pointer to the previous node and `curr` is a pointer to the current node.

Fill in the blank to obtain the address of the next node.

$$\text{Node *next} = (\text{uintptr_t})\text{prev} \wedge (\text{uintptr_t})\text{curr} \oplus \text{npv}$$

18. What does the function `func` do?

```

void push(Stack *s1, Stack *s2, int x) {
    if (isEmpty(s2) || x <= peek(s2))
        push(s2, x);
    push(s1, x);
}

int pop(Stack *s1, Stack *s2) {
    int x = pop(s1);
    if (x == peek(s2))
        pop(s2);
    return x;
}

int func(Stack *s2) {
    return peek(s2);
}

```

- (a) Returns the element that was inserted first into the stack
- (b) Implements priority queue using two stacks
- (c) Finds the maximum element in a stack using two stacks
- ☒ (d) Finds the minimum element in a stack using two stacks

19. Consider the following function:

```

int func(char expr[]) {
    Stack s;
    init(&s);

    for (int i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == '(') {
            int flag = 1;

            while (!isEmpty(&s) &&
                    peek(&s) != '(') {
                char ch = pop(&s);
                if (ch == '+' || ch == '-' ||
                    ch == '*' || ch == '/')
                    flag = 0;
            }

            pop(&s);

            if (flag)
                return 1;
        }
        else {
            push(&s, expr[i]);
        }
    }
    return 0;
}

```

Answer the following:

- (a) The value returned by the function for the input expression $(a+(b))$ is 1.
- (b) The value returned by the function for the input expression $(a+(b*c))$ is 0.
- (c) In your own words, what is the function doing?
checks redundant brackets

Indian Institute of Technology Dharwad

Data Structures and Algorithms: Version B

Quiz 1 – February 2, 2026

Roll Number: _____

Duration: 45 minutes

Max marks: 30

General Instructions:

- In questions involving code snippets with blanks, each blank corresponds to **exactly one valid statement**. Writing multiple statements in a single blank will not be evaluated.
- Answers are marked either **1** or **0**. There is **no partial marking** (e.g., no 0.5 marks).
- The **last blank sheet** of the answer booklet may be used as a rough sheet. **No additional sheets** will be provided.
- Use of mobile phones or any other **smart electronic devices** is strictly prohibited. If a student is found in possession of such devices during the examination, it will lead to **immediate expulsion from the course**.

1. Fill in the blanks so that the function correctly returns the middle node of the list.

```
Node* findMiddle(Node *head) {
    Node *slow = head;
    Node *fast = head;
    if (head == NULL)
        return NULL;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

2. The output of the below program is

```
void modify(int **p, int *q) {
    *p = q;
    **p = **p + 5;
}

int main() {
    int x = 3, y = 7;
    int *r = &x;
    modify(&r, &y);
    printf("%d %d", x, y);
    return 0;
}
```

3 12

3. What does the below function do?

```
Node* func(Node *head, int x, int y) {
    if (x == y)
        return head;

    Node *prevX = NULL, *currX = head;
    while (currX != NULL &&
           currX->data != x) {
        prevX = currX;
    }
```

```
currX = currX->next;
}

Node *prevY = NULL, *currY = head;
while (currY != NULL &&
       currY->data != y) {
    prevY = currY;
    currY = currY->next;
}

if (currX == NULL || currY == NULL)
    return head;

if (prevX != NULL)
    prevX->next = currY;
else
    head = currY;

if (prevY != NULL)
    prevY->next = currX;
else
    head = currX;

Node *temp = currX->next;
currX->next = currY->next;
currY->next = temp;

return head;
}
```

- Swaps the data values of the nodes containing x and y
 - Reverses the linked list between the nodes containing x and y
 - Swaps the positions of the nodes containing x and y by modifying links
 - Deletes the nodes containing x and y from the list
4. What is the effect of calling fun(head)?

```
void fun(struct node *p) {
    while (p->next != NULL) {
        p = p->next;
    }
```

```

    }
    p->next = head;
}

```

- (or)
- (a) The linked list becomes circular
 - (b) The last node of the list is deleted
 - (c) A duplicate of the first node is added at the end
 - (d) The program results in a segmentation fault

5. Which of the following statements is **TRUE** for the below function?

```

void func(struct node *p) {
    if (p == NULL || p->next == NULL)
        return;
    p->data = p->next->data;
    p->next = p->next->next;
}

```

- (a) The function correctly deletes any node, including the last node
- (b) The function deletes the node pointed to by p without using the head pointer
- (c) The function causes memory leakage only when p points to the first node
- (d) The function swaps data between two nodes but does not delete any node

6. Consider the following function:

```

int func(char expr[]) {
    Stack s;
    init(&s);

    for (int i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == ')') {
            int flag = 1;

            while (!isEmpty(&s) &&
                   peek(&s) != '(') {
                char ch = pop(&s);
                if (ch == '+' || ch == '-' ||
                    ch == '*' || ch == '/')
                    flag = 0;
            }

            pop(&s);

            if (flag)
                return 1;
            else {
                push(&s, expr[i]);
            }
        }
    }
    return 0;
}

```

Answer the following:

- (a) The value returned by the function for the input expression (a+(b)) is 1.

(b) The value returned by the function for the input expression (a+(b*c)) is 0.

(c) In your own words, what is the function doing?

checks redundant brackets

7. What does the below function do?

```

Node* func(Node *head, int k) {
    if (head == NULL || k == 0)
        return head;
    Node *p = head;
    int count = 1;
    while (count < k && p != NULL) {
        p = p->next;
        count++;
    }
    if (p == NULL) return head;

    Node *q = p;
    while (q->next != NULL)
        q = q->next;

    q->next = head;
    head = p->next;
    p->next = NULL;
    return head;
}

```

- (a) Rotates the singly linked list to the right by k positions
- (b) Rotates the singly linked list to the left by k positions
- (c) Reverses the singly linked list in groups of size k
- (d) Moves the last k nodes of the list to the front

8. Fill in the blanks so that the function correctly inserts the new node at the rear of the double-ended circular queue.

Constraint: Only the head pointer must be used. No temporary pointer variables are allowed.

```

Node* insertRear(Node *head, int data) {
    Node *newNode = createNode(data);
    if (head == NULL) return newNode;

    newNode->next = head;
    newNode->prev = head->prev;
    head->prev->next = newNode;
    head->prev = newNode;
    return head;
}

```

9. Consider the following function. Given that the doubly linked list is sorted in ascending order, what does the below function do?

```

Node* func(Node *head, int data) {
    Node *newNode = createNode(data);

    if (head == NULL) return newNode;
}

```



```

if (data <= head->data) {
    newNode->next = head;
    head->prev = newNode;
    return newNode;
}

Node *curr = head;
while (curr->next != NULL
    && curr->next->data < data)
    curr = curr->next;

newNode->next = curr->next;
if (curr->next != NULL)
    curr->next->prev = newNode;

curr->next = newNode;
newNode->prev = curr;

return head;
}

```

- (a) Inserts the new node immediately before the first node whose value is greater than or equal to **data**, but does not correctly update backward links
- (b) Inserts the new node after the last node whose value is less than **data**, but may violate the sorted order when duplicate values exist
- ☒ (c) Inserts a new node into the sorted doubly linked list while maintaining the sorted order
- (d) Inserts the new node at a position determined only by forward traversal, ignoring the **prev** pointers

10. Complete the following function for deleting a node from a doubly linked list using a **single pointer p**. No other pointer must be used.

```

void func(Node *p) {
    if (p == NULL) return;

    if (p->prev != NULL)
        p->prev->next = p->next;

    if (p->next != NULL)
        p->next->prev = p->prev;

    free(p);
}

```

11. Complete the following function that checks whether a doubly linked list is a palindrome:

```

int isPalindrome(Node *head) {
    if (head == NULL)
        return 1;

    Node *left = head;
    Node *right = head;

    while (right->next != NULL)

```

```

        right = right->next;

    while (left != right && right->next != left) {
        if (left->data != right->data)
            return 0;

        left = left->next;
        right = right->prev;
    }

    return 1;
}

```

12. Consider the following C program:

```

void main() {
    int arr[3] = {1, 2, 3};
    int *p = arr;
    printf("%d ", *(p + 1));
    p++;
    arr++;
    printf("%d", *p);
}

```

Which of the following is **correct**?

- (a) The program compiles and prints 2 2
- (b) The program compiles and prints 2 3
- ☒ (c) The program fails to compile
- (d) The program results in undefined behaviour at runtime
13. Consider the following function. Assume that the array represents a priority queue, where a smaller value indicates a higher priority.

```

int func(int arr[], int *n) {
    int idx = 0;

    for (int i = 1; i < *n; i++) {
        if (arr[i] < arr[idx])
            idx = i;
    }

    int item = arr[idx];

    for (int i = idx; i < *n - 1; i++)
        arr[i] = arr[i + 1];

    (*n)--;

    return item;
}

```

What does the above function do?

- (a) Deletes and returns the element inserted first into the queue
- ☒ (b) Deletes and returns the element with the highest priority from the queue
- (c) Deletes and returns the element with the lowest priority from the queue
- (d) Deletes all elements from the queue

14. Determine the output of the program.

```
#include <stdio.h>
int main() {
    int a = 5, b = 10;
    int *p = &a, *q = &b;
    *p = *q;
    q = p;
    *q = 20;
    printf("%d %d", a, b);
    return 0;
}
```

20 10

15. Assume a double ended queue (deque) is implemented using a circular array of size N . The indices **first** and **last** represent the front and rear of the queue respectively.

(a) **Insert at Front:**

Fill in the blank to correctly update the index when inserting an element at the front of a non-empty deque.

$$\text{first} = (\text{first} - 1 + N) \% N;$$

(b) **Delete from Rear:**

Fill in the blank to correctly update the index when deleting an element from the rear of a non-empty deque.

$$\text{last} = (\text{last} - 1 + N) \% N;$$

16. In a memory efficient doubly linked list, each node stores the XOR of the addresses of the previous and next nodes in a field named **npv**.

Assume **prev** is a pointer to the previous node and **curr** is a pointer to the current node.

Fill in the blank to obtain the address of the next node.

$$\text{Node *next} = (\text{uintptr_t})\text{prev} \wedge (\text{uintptr_t})\text{curr} \wedge \text{npv}$$

17. What does the below function do?

```
Node* func(Node *head, int n) {
    Node *p = head;
    Node *q = head;
    while (n--) {
        if (p == NULL)
            return NULL;
        p = p->next;
    }
    while (p != NULL) {
        p = p->next;
        q = q->next;
    }
    return q;
}
```

- (a) Returns the address of the $(n + 1)^{\text{th}}$ node from the end of the list
- (b) Returns the address of the $(n + 1)^{\text{th}}$ node from the beginning of the list
- (c) Returns the address of the $(n - 1)^{\text{th}}$ node from the end of the list
- (d) Returns the address of the $(n - 1)^{\text{th}}$ node from the beginning of the list
- (e) Returns the address of the n^{th} node from the beginning of the list
- (f) Returns the address of the n^{th} node from the end of the list

18. What does the function **func** do?

```
void push(Stack *s1, Stack *s2, int x) {
    if (isEmpty(s2) || x <= peek(s2))
        push(s2, x);
    push(s1, x);
}

int pop(Stack *s1, Stack *s2) {
    int x = pop(s1);
    if (x == peek(s2))
        pop(s2);
    return x;
}

int func(Stack *s2) {
    return peek(s2);
}
```

- (a) Returns the element that was inserted first into the stack
- (b) Implements priority queue using two stacks
- (c) Finds the maximum element in a stack using two stacks
- (d) Finds the minimum element in a stack using two stacks

19. What does the below function do?

```
Node* func(Node *head) {
    Node *prev = NULL;
    Node *curr = head;
    Node *next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

- (a) Rotates the singly linked list by one position
- (b) Deletes alternate nodes of the linked list
- (c) Finds the middle node of the linked list
- (d) Reverses the singly linked list

Indian Institute of Technology, Dharwad

Data Structures and Algorithms

Mid Sem - February 24, 2026

Instructors: Vijeth K & Kedar K

Roll No.: _____

Duration: **120 minutes**

Max Marks: **50**

Instructions:

- Assume all linked lists are dynamically allocated unless stated otherwise.
 - Use of mobile phones or any other smart electronic devices is strictly prohibited. Any student found in possession of such devices during the examination will be immediately expelled from the course.
-

1 Answer the Following (1 point each)

Q1. Consider the following My-order strategy for traversing a binary tree:?

```
Visit the root;  
Visit the right subtree using My -order  
Visit the left subtree using My -order
```

Which of the below is the My-order traversal of the expression tree corresponding to the reverse polish (postfix) expression: $3\ 4\ *\ 5\ -\ 2\ ^\ 6\ 7\ *\ 1\ +\ -$

A) $+ - 1\ 6\ 7\ *\ 2\ ^\ 5 - 3\ 4\ *$

C) $- + 1\ *\ 7\ 6\ ^\ 2 - 5\ *\ 4\ 3$

B) $- + 1\ *\ 6\ 7\ ^\ 2 - 5\ *\ 3\ 4$

D) $1\ 7\ 6\ *\ +\ 2\ 5\ 4\ 3\ *\ -\ ^\ -$

Answer: (D)

Q2. The postorder traversal of a binary tree is 8, 9, 6, 7, 4, 5, 2, 3, 1. The inorder traversal of the same tree is 8, 6, 9, 4, 7, 2, 5, 1, 3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is

A) 2

B) 3

C) 4

D) 5

Answer: (C)

Q3. In a queue implemented with a circular array of size N, where head and tail are associated with the indices of the next element to dequeue and the most recently enqueued element, respectively, which of the following expressions evaluates to True if and only if the queue is full?

A) $(tail + 1) \% N == head$

C) $tail \% N == head - 1$

B) $(head + 1) \% N == tail$

D) $(head + N) \% N == head$

Answer: (A)

Q4. Suppose you are given an implementation of a queue of integers. The operations that can be performed on the queue are:

- i. `isEmpty(Q)` returns true if the queue is empty, false otherwise.
- ii. `delete(Q)` deletes the element at the front of the queue and returns its value.
- iii. `insert(Q,i)` inserts the integer `i` at the rear of the queue. Consider the following function

```
void f(queue Q) {
    int i ;
    if (!isEmpty(Q)) {
        i = delete(Q);
        f(Q);
        insert(Q,i);
    }
}
```

What operation is performed by the above function `f`?

- A) Leaves the queue `Q` unchanged
- B) Reverses the order of the elements in the queue `Q`
- C) Deletes the element at the front of the queue `Q` and inserts it at the rear keeping the other elements in the same order
- D) Empties the queue `Q`

Answer: (B)

Q5. An implementation of a queue `Q`, using two stacks `S1` and `S2`, is given below:

```
void insert(Q,x){
    push (S1,x);
}
void delete(Q){
    if (stack-empty (S2))then
        if (stack-empty (S1))then{
            print("Q is empty");
            return;
        }
        else while (!(stack-empty)(S1)){
            x=pop(S1);
            push(S2,x);
        }
        x=pop(S2);
}
```

Let n insert and $m(\leq n)$ delete operations be performed in an arbitrary on an empty queue `Q`, Let x and y be the number of push and pop operations performed, respectively, in the processes. Which of the following is true for all m and n ?

- | | |
|--|--|
| <p>A) $m + n \leq x \leq 2n$ and $2m \leq y \leq n + m$</p> <p>B) $m + n \leq x \leq 2n$ and $2m \leq y \leq 2n$</p> | <p>C) $2m \leq x \leq 2n$ and $2m \leq y \leq n + m$</p> <p>D) $2m \leq x \leq 2n$ and $2m \leq y \leq 2n$</p> |
|--|--|

Answer: (A)

Q6. Given the infix expression:

$$A + B * (C - D) / E^F$$

Assume:

- $\wedge$ (exponent) is right associative
- Precedence: $(\wedge) > (* /) > (+ -)$

The correct postfix expression is:

Answer: A B C D - * E F $\wedge$ / +

Q7. Consider the following code:

<pre>struct node { int data; struct node *next; };</pre>	<pre>void modify(struct node *head) { struct node *p = head; while (p && p->next) { p->next = p->next->next; p = p->next; } }</pre>
--	--

If the original list is:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow \text{NULL}$

After calling `modify(head)`, the resulting list is:

Answer: $1 \rightarrow 3 \rightarrow 5 \rightarrow \text{NULL}$

Q8. Consider deletion of node X from a doubly linked list. Assume X is neither head nor tail. Which of the following pointer updates is correct and complete?

- | | |
|--|--|
| (A) <code>X->prev->next = X->next;</code> | (C) <code>X->prev->next = X->next;</code>
<code>X->next->prev = X->prev;</code> |
| (B) <code>X->next->prev = X->prev;</code> | (D) <code>X = X->next;</code> |

Answer: (C)

Q9. Consider a circular singly linked list with n nodes. You are given only a pointer `last` (pointing to the last node), where:

`last->next = first node`

If the list is traversed using:

```
struct node *p = last->next;  
int count = 0;  
do {  
    count++;  
    p = p->next;  
} while (p != last->next);
```

The value of `count` after execution will be _____
Answer: n

Q10. Consider the following function intended to delete the first node of a doubly linked list:

```
void deleteFirst(struct node **head) {  
    if (*head == NULL) return;  
  
    struct node *temp = *head;  
    *head = (*head)->next;  
    (*head)->prev = NULL;  
    free(temp);  
}
```

What is wrong with this code?

- | | |
|-----------------------------------|--|
| (A) It works fine. | (C) It does not update the next pointer correctly. |
| (B) It causes segmentation fault. | (D) It does not free the correct node. |

Answer: (B)

Explanation (students don't have to write an explanation): If the list has only one node, after `*head = (*head)->next`; `*head` becomes `NULL`, and the next line `(*head)->prev = NULL`; dereferences `NULL` causing segmentation fault.

2 Answer the Following (Q1:10 points and Q2 to Q6:2 points each)

Q1. Parts a. b. c. refer to the binary tree given in Fig.1.

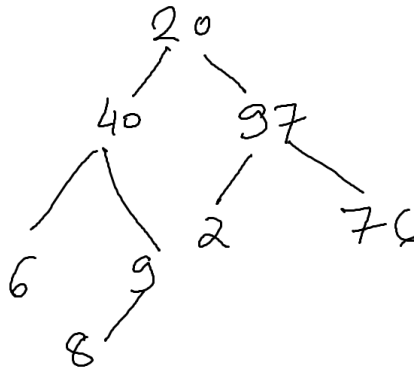


Figure 1: Binary Tree

- List the data that would be accessed by a pre-order traversal on the given tree writing out the values in the nodes as they would be accessed, separated by commas.
- List the data that would be accessed by an in-order traversal on the given tree writing out the values in the nodes as they would be accessed, separated by commas.
- List the data that would be accessed by a level-order traversal on the given tree writing out the values in the nodes as they would be accessed, separated by commas.
- In general, if a ternary tree (at most three children per node) tree is perfectly balanced (unlike the tree pictured here) and complete with height h , how many leaves in terms of h will the tree have? Note, this tree has a perfectly flat bottom.
- In general, if a binary tree is perfectly balanced (unlike the tree pictured here) and complete with exactly k leaves, what is the height of this tree (in terms of k)? Note, this tree has a perfectly flat bottom.

Answer:

- 20, 40, 6, 9, 8, 97, 2, 76
- 6, 40, 8, 9, 20, 2, 97, 76
- 20, 40, 97, 6, 9, 2, 76, 8
- 3^h
- $\log_2 k$

Q2. Consider a singly linked list. You are given a pointer to a node X (which is not the last node). Write the statements required to delete X without using the head pointer.

Answer: $X \rightarrow data = X \rightarrow next \rightarrow data;$

$temp = X \rightarrow next;$

$X \rightarrow next = X \rightarrow next \rightarrow next;$

$free(temp);$

(Core idea: copy next node's data and bypass it)

If idea is somewhat similar (close) then give 1pt otherwise, 0.

Q3. In a circular singly linked list with only a pointer to the last node:

- Write the condition to detect whether the list is empty.
- Explain how you can detect whether the list contains exactly one node.

Answer: (a) If $last == NULL$, then the list is empty. – 1 point or 0. There are no 0.5 marks

(b) If $last! = NULL$ and $last \rightarrow next == last$, then the list contains exactly one node. – 1 point or 0. There are no 0.5 marks.

Q4. Consider the following snippet

```
struct node *p = head->next;
head->next = p->next;
p->next = head;
head = p;
```

What is the snippet doing? Identify the condition(s) in which the above code fails and give correction(s) to the snippet to fix the case.

It performs an adjacent swap of the first two nodes. – 0.5 points

The code fails when: – 0.25 for each

- a) The list is empty (*head == NULL*)
- b) The list contains only one node (*head->next == NULL*)

Corrected code: – 1 point if all statements are correct

```
if(head != NULL && head->next != NULL){
structnode *p = head->next;
head->next = p->next;
p->next = head;
head = p;
}
```

Q5. In a stack implemented using a singly linked list, what condition must be checked before performing `pop()`? What happens if it is not checked?

Answer: Check if `top == NULL`. – 1 point

If not checked, dereferencing `NULL` causes segmentation fault. – 1 point

Q6. Consider the following function:

```
struct node* func(struct node *a, struct node *b) {
    struct node dummy;
    struct node *tail = &dummy;
    dummy.next = NULL;

    while (a != NULL && b != NULL) {
        if (a->data <= b->data) {
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }

    if (a != NULL)
        tail->next = a;
    else
        tail->next = b;

    return dummy.next;
}
```

Assume that the linked lists pointed to by `a` and `b` are sorted in ascending order. What does this function do? Why is `"dummy.next"` returned at the end?

Answer: It combines two sorted singly linked lists into a single sorted linked list (ascending order) by rearranging existing links. No new nodes are created. – 1 point

`dummy.next` will be the head of the merged list. – 1 point

3 Answer the following (5 points each)

Q1. Answer the below questions related to the binary search tree (BST)

- Build a BST using the following numbers 5, 10, 50, 3, 1, 4, 9, 8, 7. You can show the final tree (1pt)
- Delete 5 from the BST tree constructed in a. and draw the final tree. (2pts)
- Delete 7 from the BST tree constructed in b. and draw the final tree. (2pts)

Answer:

a. (1pt)

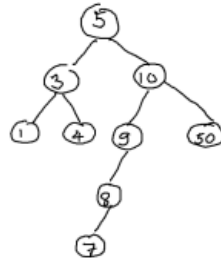


Figure 2: Binary Search Tree

b. (2pts)

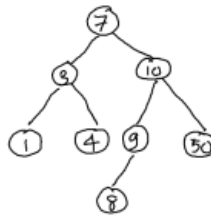


Figure 3: Binary Search Tree After deleting 5

c. (2pts)

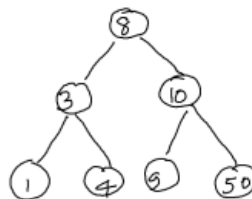


Figure 4: Binary Search Tree After deleting 7

Q2. Insert the following sequence of values into an AVL (Height Balanced Tree) tree in the given order. Show all the steps. (5pts)

4, 10, 3, 8, 5, 6, 25

Answer: (5pts)

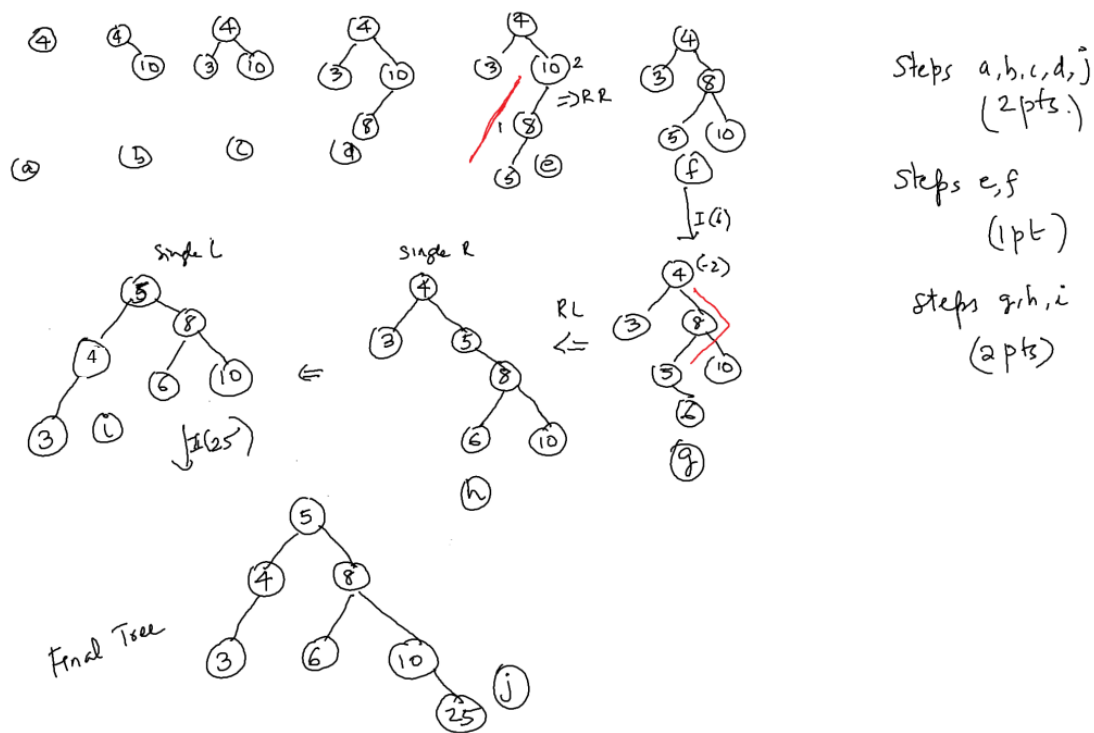


Figure 5: AVL Tree

Q3. A university wants to develop a “Smart Campus Service System” with the following functionalities:

- Students submit support requests. Requests must be handled in the order they are received.
- During peak time, emergency requests must be handled before normal requests, but within the same category, order of arrival must be maintained.
- The system must support an “Undo” feature for administrative changes (each undo reverses the most recent action).
- The university stores course information in a hierarchical form: University → School → Department → Course.
- The system must store student records such that:
 - Records are kept sorted by Student ID.
 - Searching by Student ID should be efficient.

For each functionality:

- Identify the most appropriate data structure (from Stack, Queue, Circular Queue, Priority Queue, Binary Tree, Generic Tree, BST).
- Justify your choice in one or two sentences.

Answer: 1 point each

- Queue : Because requests must be processed in FIFO order.
- Priority Queue : Because emergency requests need higher priority, while maintaining FIFO within same priority.
- Stack : Because undo follows LIFO principle.
- Generic Tree : Because hierarchy is multi-level with variable number of children.
- BST : Because records must remain sorted and searchable efficiently.

Q4. A city traffic control system uses different types of queues for different functionalities. Answer the following:

- (a) In a roundabout parking system, when the queue becomes full, new vehicles can occupy positions freed at the beginning of the queue.

(i) Which type of queue should be used?

(ii) Write the statement used to update rear in this queue (array-based).

Answer: 0.5 each

(i) Circular Queue

(ii) `rear = (rear + 1) % size;`

- (b) Ambulances must be given priority over normal vehicles at a checkpoint. However, vehicles of the same category must be processed in order of arrival.

(i) Which type of queue is appropriate?

(ii) Write a small logic snippet to insert a vehicle with priority into a linked-list-based queue.

Answer: (i) Priority Queue – 0.5 points

(ii) Full code – 1.5

```
if(head == NULL || new->priority < head->priority){
    new->next = head;
    head = new;
}else{
    temp = head;
    while(temp->next != NULL && temp->next->priority <= new->priority)
        temp = temp->next;
    new->next = temp->next;
    temp->next = new;
}
```

- (c) In a railway station platform system, VIP passengers may enter from the front of the queue while normal passengers enter from the rear.

(i) Which queue variant is suitable?

(ii) Write the operation to insert at the front in an array-based implementation.

Answer: (i) Deque (Double Ended Queue) – 0.5 points

(ii) correct logic – 1.5 points otherwise 0. No in between marks

`front = (front - 1 + size) % size;`

`q[front] = passenger;`

Indian Institute of Technology Dharwad

Data Structures and Algorithms: Version A

Quiz 2 – April 6, 2026

Roll Number: _____

Duration: 45 minutes

Max marks: 30

General Instructions:

1. In questions involving code snippets with blanks, each blank corresponds to **exactly one valid statement**. Writing multiple statements in a single blank will not be evaluated.
2. Answers are marked either **1 or 0**. There is **no partial marking** (e.g., no 0.5 marks).
3. The **last blank sheet** of the answer booklet may be used as a rough sheet. **No additional sheets** will be provided.
4. Use of mobile phones or any other **smart electronic devices** is strictly prohibited. If a student is found in possession of such devices during the examination, it will lead to **immediate expulsion from the course**.

1. A B-Tree of order m is given. (0.5 points each)

- (a) The maximum number of children a node can have is _____.
- (b) The minimum number of children (except root) a node must contain is _____.

Answer: m , $\text{ceil}(m/2)$

2. A B-Tree of order $m = 3$ is initially empty. The following keys are inserted in order:

10, 20, 5, 6, 12, 30, 7, 17, 3, 25

How many **node splits** occur during the entire insertion process?

Answer: 4

3. Fill in the blanks for a Red-Black Tree: (0.5 points each)

- (a) The longest path from the root to a leaf can be at most _____ times the length of the shortest path.
- (b) If a Red-Black Tree has n nodes, then its height is at most _____.

Answer: 2, $2 \log_2(n+1)$

4. Fill in the blanks so that the function correctly performs heapify (downward) for a max-heap.

```
void heapify(int arr[], int n, int i) {
    int largest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (_____)
        largest = l;

    if (_____)
        largest = r;

    if (largest != i) {
        int temp = arr[i];
```

```
arr[i] = arr[largest];
arr[largest] = temp;
    _____;
    }
}
```

Answer: $l < n \ \&\& \text{arr}[l] > \text{arr}[\text{largest}]$

$r < n \ \&\& \text{arr}[r] > \text{arr}[\text{largest}]$

heapify(arr, n, largest)

5. Fill in the blank so that the function correctly computes the GCD of two numbers using recursion.

```
int func(int a, int b) {
    if (b == 0)
        return a;

    return _____;
}
```

Answer: $\text{func}(b, a \% b)$

6. Determine the value returned by $\text{func}(5)$.

```
int func(int n) {
    if (n == 0)
        return 1;

    if (n % 2 == 0)
        return func(n / 2);

    return func(n - 1) + n;
}
```

_____ Answer: 7

7. What does the following function do?

```
void func(int n) {
    if (n == 0)
        return;

    func(n / 2);
    printf("%d", n % 2);
}
```

- (a) Prints the binary representation of n in reverse order
 (b) Prints the binary representation of n
 (c) Prints the decimal digits of n
 (d) Prints whether n is even or odd

Answer (b)

8. Fill in the blanks so that the function correctly checks whether there exists a subset with sum equal to a given value `target`.

```
int func(int arr[], int n, int i,
        int sum, int target) {
    if (sum == target) return 1;

    if (i == n) return 0;

    if (_____)
        return 1;

    if (_____)
        return 1;

    return 0;
}
```

Answer: `func(arr, n, i + 1, sum + arr[i], target)`
`func(arr, n, i + 1, sum, target)`

9. How many outputs are generated by the following function call `func(5, 0)`?

```
void func(int n, int i) {
    if (i == n) {
        printSolution();
        return;
    }

    func(n, i + 1);

    if (i % 2 == 0)
        func(n, i + 1);
}
```

Answer: 8

10. Determine the output of the following function call `func(3)`.

```
void func(int n) {
    if (n == 0)
        return;

    printf("%d ", n);
    func(n - 1);
    printf("%d ", n);
}
```

Answer: 3 2 1 1 2 3

- What does the following function generate?

```
void func(int arr[], int n, int i) {
    if (i == n) {
        printArray(arr, n);
        return;
    }

    for (int j = i; j < n; j++) {
        swap(arr[i], arr[j]);
        func(arr, n, i + 1);
        swap(arr[i], arr[j]);
    }
}
```

- (a) All subsets of the array
 (b) All permutations of the array
 (c) All combinations of size 2
 (d) All sorted arrangements of the array

Answer: (b)

12. Fill in the blanks: (0.5 points each)

- (a) The average time complexity of search in a hash table is _____.
 (b) In the worst case, the time complexity becomes _____.

Answer: (a) $O(1)$

(b) $O(n)$

13. Consider a hash table of size 7 with hash functions:

$$h_1(k) = k \bmod 7, \quad h_2(k) = 1 + (k \bmod 5)$$

Find the position of key 12 if collision occurs at the first position.

Answer: $h_1(12)=5; \quad h_2(12)=1+(2)=3 \quad \text{final index} = (5+3) \bmod 7 = 1$

14. Consider a hash table of size 7 with hash function

$$h(k) = k \bmod 7$$

Keys 10, 17, 24 are inserted using **quadratic probing** with probe sequence:

$$h(k) + i^2$$

Fill in the final positions of the keys in the hash table.

[____, ____, ____, ____, ____, ____, ____]

Answer: [24, , , 10, 17, ,]

15. Consider a hash table of size 11 with hash function

$$h(k) = k \bmod 11$$

Keys are inserted using **linear probing**.

The keys 22, 1, 13, 11, 24, 33 are inserted in this order.

The **final position** of key 33 is _____

Answer: 5

16. Determine the BFS traversal starting from vertex A for the following graph.
- Adjacency list:
- A: B, C
B: D, E
C: F
D:
E: F
F:
-
- Answer: A B C D E F
17. Determine the DFS traversal starting from vertex A for the following graph.
- Adjacency list:
- A: B, C
B: D, E
C: F
D:
E: F
F:
-
- Answer: A B D E F C
18. Which of the following statements is **TRUE**?
- (a) BFS uses a stack for traversal
(b) DFS always finds the shortest path in a graph
(c) BFS explores nodes level by level
(d) DFS cannot be implemented recursively
- Answer: BFS explores nodes level by level
19. Fill in the blank so that the function correctly performs BFS traversal.
- ```
void BFS(int start) {
 _____;
 visited[start] = 1;

 while (!isEmpty()) {
 int v = _____;
 visit(v); //Process v

 for (each neighbor u of v) {
 if (!visited[u]) {
 visited[u] = 1;
 _____;
 }
 }
 }
}
```
- Answer: enqueue(start); dequeue(); enqueue(u);
20. In Dijkstra's algorithm, suppose the current node is  $u$  and we are relaxing an edge  $u \rightarrow v$  with weight  $w$ . Fill in the blank for updating the distance of  $v$ :
- if ( $\text{dist}[v] > \underline{\hspace{2cm}}$ )
- Answer:  $\text{dist}[u] + w$
21. Which of the following conditions is required for Dijkstra's algorithm to work correctly?
- (a) Graph must be directed  
(b) Graph must not contain negative edge weights  
(c) Graph must be acyclic  
(d) Graph must be complete
- Answer: Graph must not contain negative edge weights
22. Consider the following weighted graph:
- A --3--> B  
A --1--> C  
B --4--> D  
B --2--> E  
C --2--> B  
C --5--> D  
C --8--> F  
D --1--> E  
E --3--> F
- Dijkstra's algorithm is executed starting from source vertex A. After **4 iterations** (including the source), which vertices have been marked as visited? Write the vertices in the order their visit.
- 
- Answer: A C B E
23. Which of the following statements is **TRUE** for the Bellman-Ford algorithm?
- (a) It cannot handle negative edge weights  
(b) It detects negative weight cycles in a graph  
(c) It always runs faster than Dijkstra's algorithm  
(d) It requires the graph to be acyclic
- Answer: It detects negative weight cycles in a graph
24. In a graph with  $V$  vertices, how many times are all edges relaxed in the Bellman-Ford algorithm in the worst case?
- 
- Answer:  $V - 1$
25. All-pairs shortest paths in a graph are computed by running the Bellman-Ford algorithm from every vertex as the source.
- If the graph has  $V$  vertices and  $E$  edges, what is the overall time complexity of this approach?
- 
- Answer:  $O(V^2E)$

# Indian Institute of Technology Dharwad

## Data Structures and Algorithms: Version B

### Quiz 2 – April 6, 2026

Roll Number: \_\_\_\_\_

Duration: 45 minutes

Max marks: 30

#### General Instructions:

1. In questions involving code snippets with blanks, each blank corresponds to **exactly one valid statement**. Writing multiple statements in a single blank will not be evaluated.
2. Answers are marked either **1 or 0**. There is **no partial marking** (e.g., no 0.5 marks).
3. The **last blank sheet** of the answer booklet may be used as a rough sheet. **No additional sheets** will be provided.
4. Use of mobile phones or any other **smart electronic devices** is strictly prohibited. If a student is found in possession of such devices during the examination, it will lead to **immediate expulsion from the course**.

1. Consider a hash table of size 7 with hash function

$$h(k) = k \bmod 7$$

Keys 10, 17, 24 are inserted using **quadratic probing** with probe sequence:

$$h(k) + i^2$$

Fill in the final positions of the keys in the hash table.

[ \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_, \_\_\_\_\_ ]

Answer: [24, , , 10, 17, , ]

2. Determine the BFS traversal starting from vertex A for the following graph.

Adjacency list:

A: B, C

B: D, E

C: F

D:

E: F

F:

Answer: A B C D E F

3. Determine the DFS traversal starting from vertex A for the following graph.

Adjacency list:

A: B, C

B: D, E

C: F

D:

E: F

F:

Answer: A B D E F C

4. Fill in the blanks for a Red-Black Tree: (0.5 points each)

(a) The longest path from the root to a leaf can be at most \_\_\_\_\_ times the length of the shortest path.

(b) If a Red-Black Tree has  $n$  nodes, then its height is at most \_\_\_\_\_.

Answer: 2,  $2 \log_2(n + 1)$

5. Which of the following conditions is required for Dijkstra's algorithm to work correctly?

(a) Graph must be directed

(b) Graph must not contain negative edge weights

(c) Graph must be acyclic

(d) Graph must be complete

Answer: Graph must not contain negative edge weights

6. Consider the following weighted graph:

A --3--> B

A --1--> C

B --4--> D

B --2--> E

C --2--> B

C --5--> D

C --8--> F

D --1--> E

E --3--> F

Dijkstra's algorithm is executed starting from source vertex A. After **4 iterations** (including the source), which vertices have been marked as visited? Write the vertices in the order their visit.

Answer: A C B E

7. Which of the following statements is **TRUE** for the Bellman-Ford algorithm?

(a) It cannot handle negative edge weights

(b) It detects negative weight cycles in a graph

(c) It always runs faster than Dijkstra's algorithm

(d) It requires the graph to be acyclic

Answer: It detects negative weight cycles in a graph

8. In a graph with  $V$  vertices, how many times are all edges relaxed in the Bellman-Ford algorithm in the worst case?

Answer:  $V - 1$

9. All-pairs shortest paths in a graph are computed by running the Bellman-Ford algorithm from every vertex as the source.

If the graph has  $V$  vertices and  $E$  edges, what is the overall time complexity of this approach?

Answer:  $O(V^2E)$

10. Fill in the blanks so that the function correctly checks whether there exists a subset with sum equal to a given value **target**.

```
int func(int arr[], int n, int i,
 int sum, int target) {
 if (sum == target) return 1;

 if (i == n) return 0;

 if (_____)
 return 1;

 if (_____)
 return 1;

 return 0;
}
```

Answer: `func(arr, n, i + 1, sum + arr[i], target)`

`func(arr, n, i + 1, sum, target)`

11. How many outputs are generated by the following function call `func(5, 0)`?

```
void func(int n, int i) {
 if (i == n) {
 printSolution();
 return;
 }

 func(n, i + 1);

 if (i % 2 == 0)
 func(n, i + 1);
}
```

Answer: 8

12. Determine the output of the following function call `func(3)`.

```
void func(int n) {
 if (n == 0)
 return;

 printf("%d ", n);
 func(n - 1);
 printf("%d ", n);
}
```

Answer: 3 2 1 1 2 3

13. What does the following function generate?

```
void func(int arr[], int n, int i) {
 if (i == n) {
 printArray(arr, n);
 return;
 }

 for (int j = i; j < n; j++) {
 swap(arr[i], arr[j]);
 func(arr, n, i + 1);
 swap(arr[i], arr[j]);
 }
}
```

- (a) All subsets of the array
- (b) All permutations of the array
- (c) All combinations of size 2
- (d) All sorted arrangements of the array

Answer: (b)

14. Fill in the blanks: (0.5 points each)

- (a) The average time complexity of search in a hash table is \_\_\_\_\_.
- (b) In the worst case, the time complexity becomes \_\_\_\_\_.

Answer: (a)  $O(1)$  (b)  $O(n)$

15. Consider a hash table of size 7 with hash functions:

$$h_1(k) = k \bmod 7, \quad h_2(k) = 1 + (k \bmod 5)$$

Find the position of key 12 if collision occurs at the first position.

Answer:  $h_1(12)=5$ ;  $h_2(12)=1+(2)=3$

final index =  $(5+3) \bmod 7 = 1$

16. Consider a hash table of size 11 with hash function

$$h(k) = k \bmod 11$$

Keys are inserted using **linear probing**.

The keys 22, 1, 13, 11, 24, 33 are inserted in this order.

The **final position** of key 33 is \_\_\_\_\_

Answer: 5

17. A B-Tree of order  $m$  is given. (0.5 points each)

- (a) The maximum number of children a node can have is \_\_\_\_\_.
- (b) The minimum number of children (except root) a node must contain is \_\_\_\_\_.

Answer:  $m$ ,  $\text{ceil}(m/2)$

18. A B-Tree of order  $m = 3$  is initially empty. The following keys are inserted in order:

10, 20, 5, 6, 12, 30, 7, 17, 3, 25

How many **node splits** occur during the entire insertion process?

\_\_\_\_\_ Answer: 4

19. Which of the following statements is **TRUE**?

- (a) BFS uses a stack for traversal
- (b) DFS always finds the shortest path in a graph
- (c) BFS explores nodes level by level
- (d) DFS cannot be implemented recursively

Answer: BFS explores nodes level by level

20. Fill in the blank so that the function correctly performs BFS traversal.

```
void BFS(int start) {
 _____;
 visited[start] = 1;

 while (!isEmpty()) {
 int v = _____;
 visit(v); //Process v

 for (each neighbor u of v) {
 if (!visited[u]) {
 visited[u] = 1;
 _____;
 }
 }
 }
}
```

Answer: enqueue(start); dequeue(); enqueue(u);

21. In Dijkstra's algorithm, suppose the current node is  $u$  and we are relaxing an edge  $u \rightarrow v$  with weight  $w$ .

Fill in the blank for updating the distance of  $v$ :

if (dist[v] > \_\_\_\_\_)

Answer: dist[u] + w

22. Determine the value returned by func(5).

```
int func(int n) {
 if (n == 0)
 return 1;

 if (n % 2 == 0)
 return func(n / 2);

 return func(n - 1) + n;
}
```

\_\_\_\_\_ Answer: 7

23. What does the following function do?

```
void func(int n) {
 if (n == 0)
 return;

 func(n / 2);
 printf("%d", n % 2);
}
```

- (a) Prints the binary representation of  $n$  in reverse order
- (b) Prints the binary representation of  $n$
- (c) Prints the decimal digits of  $n$
- (d) Prints whether  $n$  is even or odd

Answer (b)

24. Fill in the blanks so that the function correctly performs heapify (downward) for a max-heap.

```
void heapify(int arr[], int n, int i) {
 int largest = i;
 int l = 2*i + 1;
 int r = 2*i + 2;

 if (_____)
 largest = l;

 if (_____)
 largest = r;

 if (largest != i) {
 int temp = arr[i];
 arr[i] = arr[largest];
 arr[largest] = temp;

 _____;
 }
}
```

Answer:  $l < n$  && arr[l] > arr[largest]

$r < n$  && arr[r] > arr[largest]

heapify(arr, n, largest)

25. Fill in the blank so that the function correctly computes the GCD of two numbers using recursion.

```
int func(int a, int b) {
 if (b == 0)
 return a;

 return _____;
}
```

Answer: func(b, a % b)



# Indian Institute of Technology, Dharwad

## Data Structures and Algorithms

End Sem - April 27, 2026

Instructors: Vijeth K & Kedar K

Roll No.: \_\_\_\_\_

Duration: **3 hours**

Max Marks: **80**

---

### Instructions:

- All the answers (including MCQs and Fill in the blanks) to be written on the answer sheets **ONLY**.
  - Assume all linked lists are dynamically allocated unless stated otherwise.
  - Each underline (\_\_\_\_) represents exactly one valid statement or expression.
  - Use of mobile phones or any other smart electronic devices is strictly prohibited. Any student found in possession of such devices during the examination will be immediately expelled from the course.
- 

## 1 Answer the following (one mark each)

**Q1.** Consider an array representation of an  $n$  element binary heap where the elements are stored from index 1 to  $n$  of the array. For the element stored at index  $i$  of the array ( $i \leq n$ ), the index of the parent is

- A)  $i-1$                       B)  $\lfloor i/2 \rfloor$                       C)  $\lceil i/2 \rceil$                       D)  $(i+1)/2$

ANS: B

**Q2.** Let LASTPOST, LASTIN, and LASTPRE denotes the last vertex visited in a **postorder**, **inorder** and **preorder** traversal respectively, of a completely binary tree. Which of the following is always true?

- A) LASTIN = LASTPOST                      C) LASTPRE = LASTPOST  
B) LASTIN = LASTPRE                      D) None of the Above

ANS:D

**Q3.** Consider the following functions:

- a)  $f(n) = 3n^{\sqrt{n}}$   
b)  $g(n) = 2^{\sqrt{n} \log_2 n}$   
c)  $h(n) = n!$

Which of the following is always true?

- A)  $h(n)$  is  $O(f(n))$                       B)  $h(n)$  is  $O(g(n))$                       C)  $g(n)$  is  $O(f(n))$                       D)  $f(n)$  is  $O(g(n))$

ANS:D

**Q4.** Consider a sequence  $a$  of elements  $a_0 = 1$ ,  $a_1 = 5$ ,  $a_2 = 7$ ,  $a_3 = 8$ ,  $a_4 = 9$ , and  $a_5 = 2$ . The following operations are performed on a stack  $S$  and a queue  $Q$ , both of which are initially empty.

I push the elements of  $a$  from  $a_0$  to  $a_5$  in that order into  $S$   
II enqueue the elements of  $a$  from  $a_0$  to  $a_5$  in that order into  $Q$   
III pop an element from  $S$   
IV dequeue an element from  $Q$   
V pop an element from  $S$   
VI dequeue an element from  $Q$   
VII dequeue an element from  $Q$  and push the same element into  $S$   
VIII Repeat operation VII three times  
IX pop an element from  $S$   
X pop an element from  $S$

The top element of  $S$  after executing the above operations is

- A) 6                                      B) 7                                      C) 8                                      D) 9

ANS: C

**Q5.** Let  $S$  be a stack of size  $n \geq 1$ . Starting with the empty stack, suppose we **PUSH** the first  $n$  natural numbers in sequence, and then perform  $n$  **POP** operations. Assume that **PUSH** and **POP** operations take  $X$  seconds each, and  $Y$  seconds elapse between the end of one such stack operation and the start of the next operation. For  $m \geq 1$ , define the stack-life of  $m$  as the time elapsed from the end of **PUSH**( $m$ ) to the start of the **POP** operation that removes  $m$  from  $S$ . The average stack-life of an element of this stack is

- A)  $n(X + Y)$                                       B)  $3Y + 2X$                                       C)  $Y + 2X$                                       D)  $n(X + Y) - X$

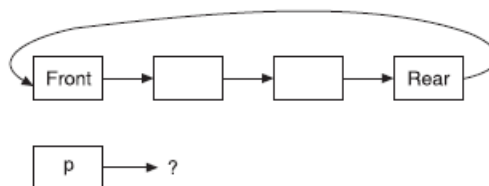
ANS: D

**Q6.** A single array  $A[1 \dots \text{MAXSIZE}]$  is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables **top1** and **top2** ( $\text{top1} < \text{top2}$ ) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for 'stack full' is

- A)  $\text{top1} = \text{top2} - 1$                                       C)  $\text{top1} + \text{top2} = \text{MAXSIZE}$   
 B)  $(\text{top1} = \text{MAXSIZE}/2)$  and  $(\text{top2} = \text{MAXSIZE}/2 + 1)$                                       D)  $(\text{top1} = \text{MAXSIZE}/2)$  or  $(\text{top2} = \text{MAXSIZE})$

ANS: A

**Q7.** A circularly linked list is used to represent a queue. A single variable  $p$  of the type **node** is used to access the queue. To which **node** should  $p$  point such that both the operations **enqueue** and **dequeue** can be performed in constant time?



- A) Front node                                      C) Not possible with single pointer  
 B) Rear node                                      D) Node next to front

ANS: B

**Q8.** A **Priority-Queue** is implemented as a **Max-Heap**. Initially, it has 5 elements. The **level order** traversal of the heap is given below: 10, 8, 5, 3, 2. Two new elements 1 and 7 are inserted in the heap in that order. The **level-order** traversal of the heap after the insertion of the elements is:

- A) 10, 8, 7, 5, 3, 2, 1                                      C) 10, 8, 7, 2, 3, 1, 5  
 B) 10, 8, 7, 3, 2, 1, 5                                      D) 10, 8, 7, 1, 2, 3, 5

ANS: B

**Q9.** Let  $Q$  denote a queue containing sixteen numbers and  $S$  be an empty stack. **Head**( $Q$ ) returns the element at the head of the queue  $Q$  without removing it from  $Q$ . Similarly **Top**( $S$ ) returns the element at the top of  $S$  without removing it from  $S$ . Consider the algorithm given below. The maximum possible number of iterations of the while loop in the algorithm is \_\_\_\_\_

```
while Q is not Empty do
 if S is Empty OR Top(S) <= Head (Q) then
 x:= Dequeue (Q);
 Push (S, x);
 else
 x:= Pop(S);
 Enqueue (Q, x);
 end
end
```



Answer:  $s[\text{index}] = '\backslash 0'$  ;  $s[\text{index}] = '1'$ ; (0.5 marks each)

**Q15.** Consider the following array:

[5, 2, 4, 6, 1, 3]

After **two passes** of Bubble Sort (sorting in ascending order), what will be the array?

A) [2, 4, 1, 3, 5, 6]

B) [2, 4, 5, 1, 3, 6]

C) [2, 1, 3, 4, 5, 6]

D) [1, 2, 3, 4, 5, 6]

Answer: A

## 2 Answer the following (two marks each)

**Q1.** Let  $p$  be a pointer as shown in the Figure 2 in a singly linked list.

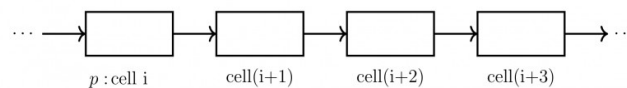


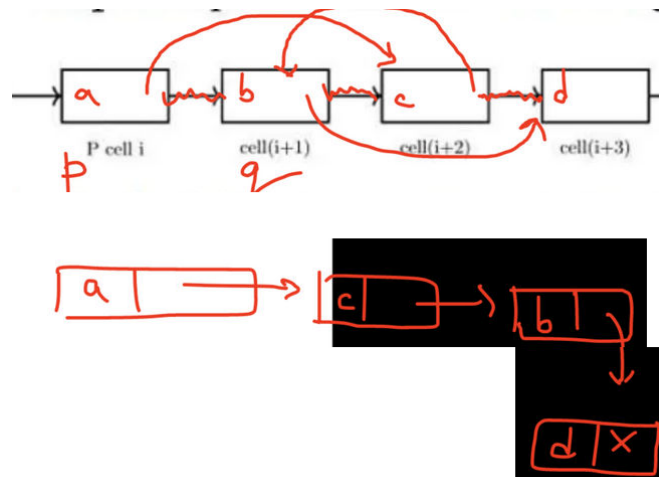
Figure 1: Linked List

What do the following assignment statements achieve?

```
q = p ->next
p->next = q->next
q->next = (q->next)->next
(p->next)->next = q
```

Ans:

The code swaps the two nodes  $\text{cell}(i+1)$  and  $\text{cell}(i+2)$  that follow the node pointed to by  $p$  cell  $(i)$ . If the explanation with cells as values is correct, award the marks. For example,



**Q2.** Given a doubly linked list where each node has two references (**prev** and **next**): one that points to a previous node and another that points to a next node. Assume the linked list in Figure 2.

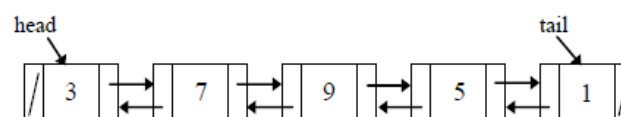


Figure 2: Doubly Linked List

and provide the output for the following code fragments. The list is restored to its initial state before each line executes:

1.  $\text{head} \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{data}$ ; \_\_\_\_\_

2. tail→prev→prev→prev→prev→next→next→data; \_\_\_\_\_

Ans:

1. 7 2. 9

**Q3.** Assume the singly linked list in Figure 3.

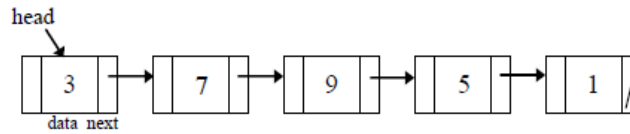


Figure 3: Linked List

Write the statements to insert a new node 10 between the node with the 5 and the node with the 1. Do not write the whole method, but just a few statements to make the connection. A new node is created with below statement:

```
node* create(int data) {
 node *tmp = (node*) malloc(sizeof(node));
 tmp->data = data;
 tmp->next = NULL;
}
```

Ans:

```
node *aux = head->next->next->next;
tmp->next = aux->next;
aux->next = tmp;
```

Alternative solution where they write a function is also acceptable provided it is correct for example:

```
void InsertAfter(node *head, int prevNode, int currNode)
{
 node *newNode = create(currNode);
 int flag = 0;
 node *tmp = head;
 while(tmp != NULL)
 {
 if (tmp->data == prevNode)
 {
 flag = 1;
 break;
 }
 tmp = tmp->next;
 }
 if(flag == 1)
 {
 node *tmp2 = tmp->next;
 tmp->next = newNode;
 newNode->next = tmp2;
 }
}
```

**Q4.** Consider the following undirected graph with vertices  $\{A, B, C, D, E, F\}$  and edges:

$$(A, B), (A, C), (B, D), (B, E), (C, F)$$

Starting from vertex A, perform:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)

Assume that adjacent vertices are visited in **alphabetical order**. Give BFS and DFS of the above graph.

Answer: BFS: A, B, C, D, E, F

DFS: A, B, D, E, C, F

**Q5.** The following code snippet is part of Kruskal's algorithm for finding the Minimum Spanning Tree (MST):

```
for each edge (u, v) in increasing order of weight {
 if (_____) {
 add edge (u, v) to MST;

 }
}
```

Fill in the blank to ensure that the algorithm correctly avoids cycles.

Answer: `find(u) != find(v)`

`unionSet(find(u), find(v))`

**Q6.** Consider the following C function:

```
#include <stdio.h>

void fun(int n, int x) {
 if (n == 0) {
 printf("%d ", x);
 return;
 }

 fun(n - 1, x + n);
 fun(n - 1, x);
}
```

What is the output of `fun(2, 0)`? Show the steps involved. `fun(2,0)` (1.5 marks for proper working steps)

```
└─ fun(1, 0+2=2)
 └─ fun(0, 2+1=3) → print 3
 └─ fun(0, 2) → print 2
└─ fun(1, 0)
 └─ fun(0, 0+1=1) → print 1
 └─ fun(0, 0) → print 0
```

Answer: 3 2 1 0 (0.5 marks)

**Q7.** Fill in the blanks to correctly implement Insertion Sort.

```
void insertionSort(int arr[], int n) {
 int i, key, j;

 for (i = 1; i < n; i++) {
 key = arr[i];
 j = i - 1;

 while (_____) {
 arr[j + 1] = arr[j];
 j = j - 1;
 }

 _____;
 }
}
```

Answer: `while (j >= 0 && arr[j] > key)`

`arr[j + 1] = key;`

### 3 Answer the following (three marks each)

**Q1.** Construct a binary tree whose preorder and inorder sequences are A, B, M, H, E, O, C, P, G, J, D, K, L, I, N, F and H, M, C, O, E, B, A, G, P, K, L, D, I, N, J, F respectively, where A, B, C, D, E, ..... are the labels of the tree nodes. Before showing the final binary tree, you need to show at least 2 partially constructed trees having all the alphabets such that they appear on the left and right sides of the final binary tree.

Ans: See Figure. Let them show Final Tree 2 mks, partial trees: 0.5+0.5

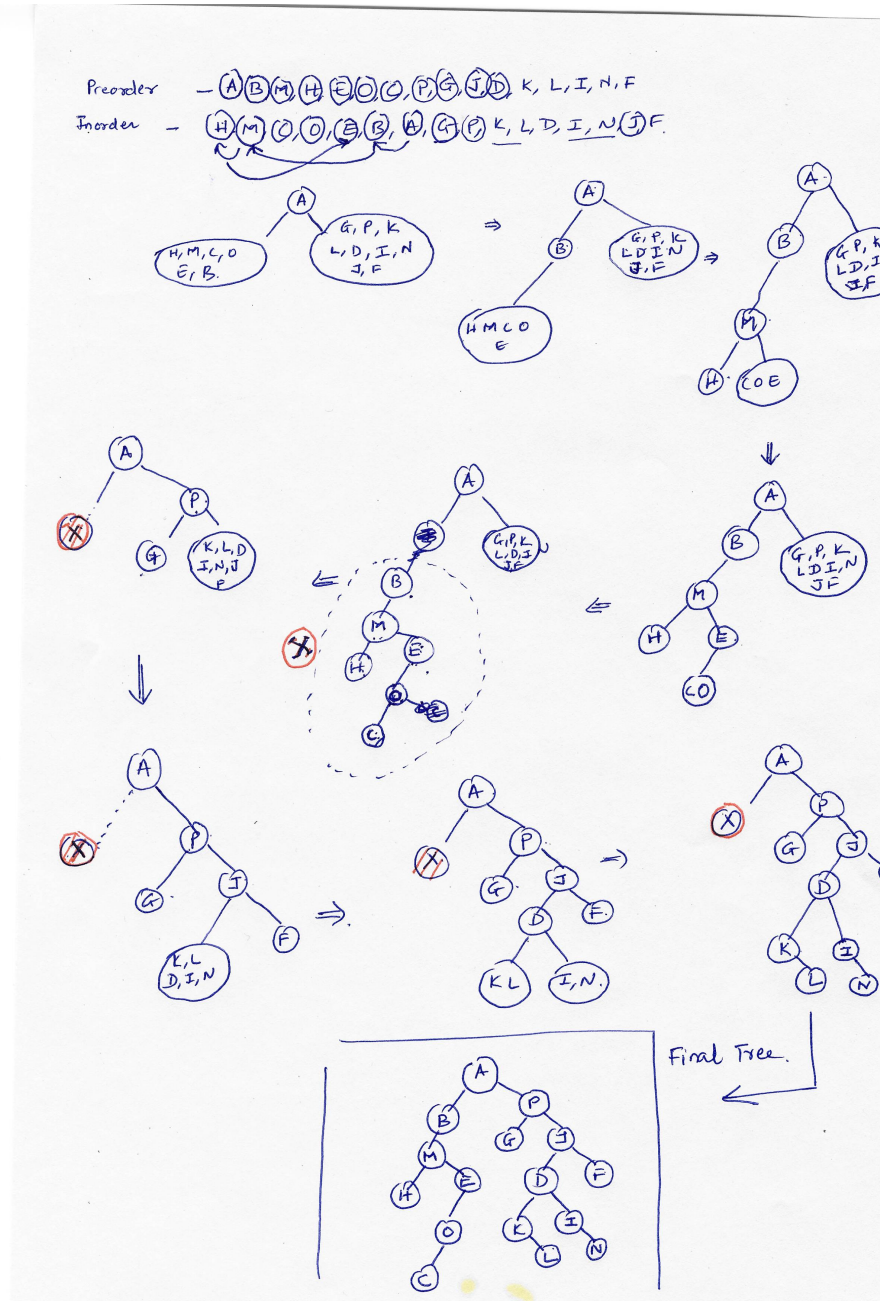


Figure 4: Caption

**Q2.** Complete the statements S1, S2 and S3 in the following function so that the function computes the depth of a binary tree rooted at t.

```
typedef struct tnode {
 int key;
 struct tnode *left, *right;
} *Tree;

int depth (Tree *t)
{
```



```

int x,y;
if (t == NULL) return 0;
x = depth(t->left);
S1: _____;
S2: if(x > y) return _____;
S3: else return _____;
}

```

ANS:

$S1: y = \text{depth}(t \rightarrow \text{right});$   
 $S2: \text{return}(x + 1);$   
 $S3: \text{return}(y + 1);$

**Q3.** Given a min-heap, following is a C program for finding the maximum element. What are S1 S2 and S3?

```

#include <stdio.h>

int findMax(int heap[], int n) {
 int i;
 int max = heap[S1];

 for (i = S2 ; i < S3 ; i++) {
 if (heap[i] > max) {
 max = heap[i];
 }
 }
 return max;
}

int main() {
 int heap[] = {1, 3, 5, 7, 9, 6, 8};
 int n = sizeof(heap) / sizeof(heap[0]);

 printf("Maximum element: %d\n", findMax(heap, n));
 return 0;
}

```

ANS

$S1: n/2$   
 $S2: n/2 + 1$   
 $S3: n$

**Q4.**

(a) Fill in the blanks in the following pseudocode for Prim's Algorithm:

```

Initialize all vertices as not visited
Select a starting vertex u
Mark u as visited

while (_____) {
 find the minimum weight edge (x, y) such that
 x is visited and y is not visited

 add edge (x, y) to MST

}

```

(b) Apply Prim's Algorithm starting from vertex  $A$  and show the steps (diagrammatically) selected to form the MST for following weighted undirected graph:

$(A, B, 2), (A, C, 3), (B, C, 1), (B, D, 4), (C, D, 5), (C, E, 6), (D, E, 7)$

Answer: (a) while (there exists an unvisited vertex) (0.5 marks)

mark  $y$  as visited (0.5 marks)

(b)  $(A, B), (B, C), (B, D), (C, E)$  (0.5 marks for each step shown)

**Q5.** Consider a disjoint set represented using the following parent array (Indexing starts from 1. A value of  $-1$  indicates a representative.):

parent =  $\{-1, 1, 1, 2, 3, 4, 5\}$

(a) What value is returned by `find(7)` (without path compression)? (1 mark)

(b) Fill in the blank to implement `pathCompressionFind()`:

```
int pathCompressionFind(int x) {
 if (parent[x] == -1)
 return x;
 return _____;
}
```

(c) After executing `pathCompressionFind(7)`, what will the updated parent array be? (1 mark)

Answer: (a) 1

(b) `parent[x] = pathCompressionFind(parent[x])`

(c) parent =  $\{-1, 1, 1, 2, 1, 4, 1\}$

**Q6.** Consider the following C function:

```
#include <stdio.h>

void fun(int arr[], int n, int index, int sum) {
 if (index == n) {
 if (sum == 3)
 printf("YES ");
 else
 printf("NO ");
 return;
 }

 fun(arr, n, index + 1, sum + arr[index]);
 fun(arr, n, index + 1, sum);
}
```

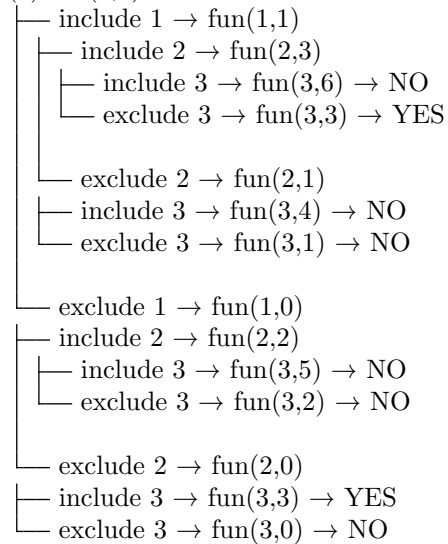
Let `arr = {1, 2, 3}` and the function is called as `fun(arr, 3, 0, 0)`.

(a) What is the output? Show proper steps. (2 marks)

(b) What does this function compute in general? (1 mark)

Answer:

(a) fun(0,0)



(b) Checks all subsets and prints whether subset sum equals 3 (subset sum problem)

**Q7.** Consider the following array:

[8, 3, 1, 7, 0, 10, 2]

The first step of Quick Sort uses the **first element as pivot** and applies the standard partition procedure.

(a) Show the array after the first partition.

(1 marks)

(b) Fill in the blank in the Quick Sort code:

```

void quickSort(int arr[], int low, int high) {
 if (low < high) {
 int p = partition(arr, low, high);
 -----;
 -----;
 }
}

```

(2 mark)

Answer:

(a) [2, 3, 1, 7, 0, 8, 10]

(b) quickSort(arr, low, p - 1)

quickSort(arr, p + 1, high);

#### 4 Answer the following (five marks each)

**Q1.** In an empty BTree of order 5, insert the following letters A, G, F, B, K, D, H, M, J, E, S, I, R, X, C, L, N, T, U, P in sequential order. Order 5 means that a node can have a maximum of 5 children and 4 keys. Show clearly pre-branching and post-branching BTree. Ans:

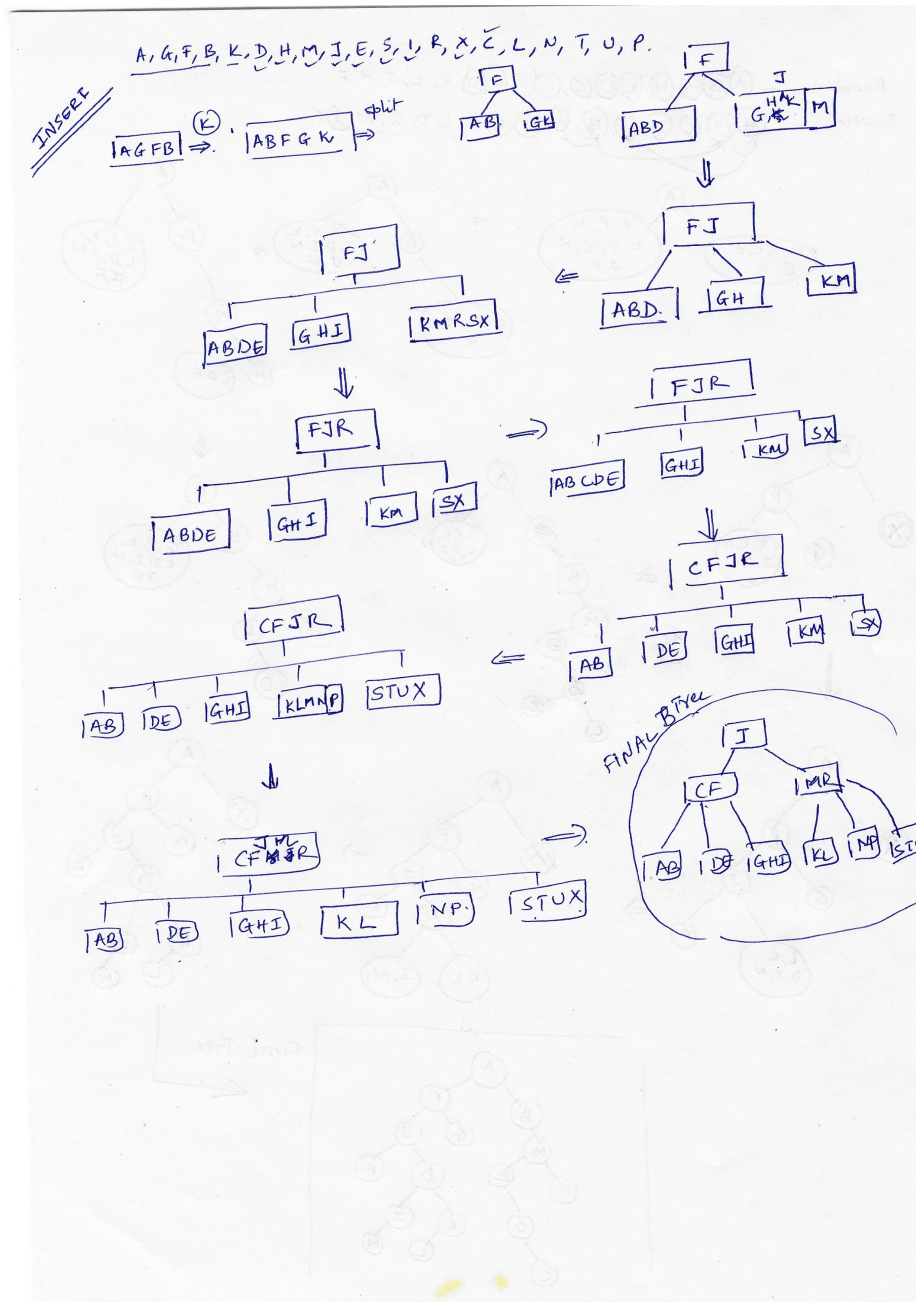


Figure 5: Caption

**Q2.** To the BTree obtained in **Q1**, delete H, R, P, D in the sequential order. Show the BTree after each letter is deleted.

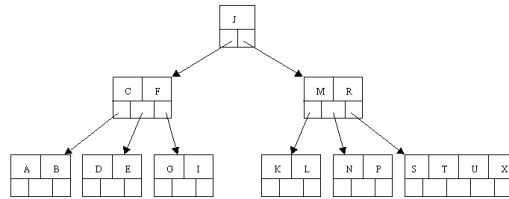
Ans:

**Q3.** Fill the following code snippet to reverse the linked list in pairs i.e. if you have a linked list that holds  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow Z$ , then after the function has been called, the linked list would hold  $2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow Z$ .

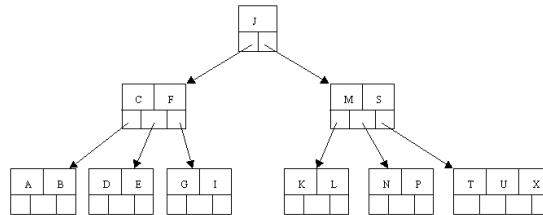
```
struct Node {
 int data;
 struct Node* next;
};

struct Node* reversePairs(struct Node* head) {
 if (____S1____ || ____S2____)
```

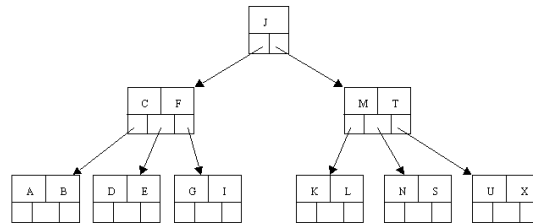
1. Delete H



2. Delete R



3. Delete P



4. Delete D

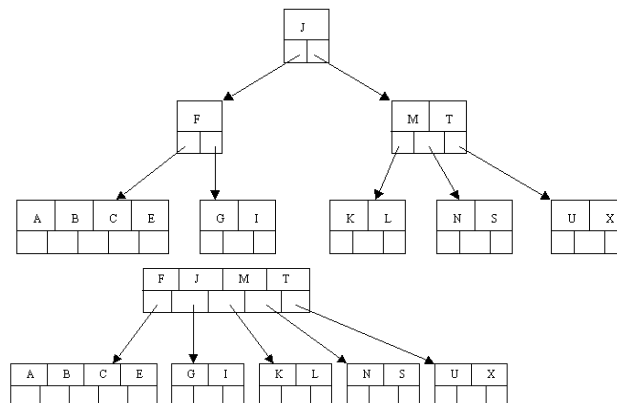


Figure 6: Caption

```

return head;

struct Node* prev = NULL;
struct Node* curr = head;
struct Node* newHead = head->next;

while (curr != NULL && curr->next != NULL) {

 struct Node* nextPair = ____S3____;
 struct Node* second = ____S4____;

 second->next = ____S5____;

```

```

 curr->next = nextPair;

 if (prev != NULL)
 prev->next = second;

 prev = curr;
 curr = nextPair;
}

return newHead;
}

```

Ans:

S1: *head == NULL*

S2: *head->next == NULL*

S3: *curr->next->next;*

S4: *curr->next;*

S5: *curr;*

**Q4.** Consider the following weighted undirected graph:

|   | A        | B        | C        | D        | E        | F        | G        |
|---|----------|----------|----------|----------|----------|----------|----------|
| A | 0        | 4        | 2        | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| B | 4        | 0        | 1        | 5        | $\infty$ | $\infty$ | $\infty$ |
| C | 2        | 1        | 0        | 8        | 10       | $\infty$ | $\infty$ |
| D | $\infty$ | 5        | 8        | 0        | 2        | 6        | $\infty$ |
| E | $\infty$ | $\infty$ | 10       | 2        | 0        | 3        | 1        |
| F | $\infty$ | $\infty$ | $\infty$ | 6        | 3        | 0        | 7        |
| G | $\infty$ | $\infty$ | $\infty$ | $\infty$ | 1        | 7        | 0        |

(a) Apply **Dijkstra's Algorithm** starting from vertex *A* and compute the shortest distance to all other vertices. Show all intermediate steps. (4 marks)

(b) If the graph were **unweighted**, what is the most efficient algorithm to find shortest paths? (1 mark)

Answer: (a) Stepwise distances:

Init:  $A = 0, B = \infty, C = \infty, D = \infty, E = \infty, F = \infty, G = \infty$

1) Pick *A* → update:  $B=4, C=2 \rightarrow A = 0, B = 4, C = 2, D = \infty, E = \infty, F = \infty, G = \infty$

2) Pick *C* → update:  $B=3, D=10, E=12 \rightarrow A = 0, B = 3, C = 2, D = 10, E = 12, F = \infty, G = \infty$

3) Pick *B* → update:  $D=8 \rightarrow A = 0, B = 3, C = 2, D = 8, E = 12, F = \infty, G = \infty$

4) Pick *D* → update:  $E=10, F=14 \rightarrow A = 0, B = 3, C = 2, D = 8, E = 10, F = 14, G = \infty$

5) Pick *E* → update:  $F=13, G=11 \rightarrow A = 0, B = 3, C = 2, D = 8, E = 10, F = 13, G = 11$

6) Pick *G* → no change  $\rightarrow A = 0, B = 3, C = 2, D = 8, E = 10, F = 13, G = 11$

7) Pick *F* → no change  $A = 0, B = 3, C = 2, D = 8, E = 10, F = 13, G = 11$

(b) Breadth-First Search (BFS)

**Q5.** Consider the following C function:

```

void swap(int *a, int *b) {
 int t = *a;
 *a = *b;
 *b = t;
}

void fun(int arr[], int l, int r) {
 if (l == r) {
 if (arr[0] + arr[1] == 3)
 printf("%d%d%d ", arr[0], arr[1], arr[2]);
 return;
 }

 for (int i = l; i <= r; i++) {
 swap(&arr[l], &arr[i]);
 fun(arr, l + 1, r);
 swap(&arr[l], &arr[i]);
 }
}

```

The function is called as:

```
int arr[] = {1,2,3}; fun(arr, 0, 2);
```

- (a) Write all outputs printed by the function. Show the steps. (4 marks)  
 (b) What does the function compute in general? (1 mark)

Answer: (a) 123 132 213 231 312 321 → but only those satisfying  $\text{arr}[0] + \text{arr}[1] == 3$ :

Steps:

```
fun([1,2,3], l=0)
├── i=0 → swap(0,0) → [1,2,3]
│ └── fun([1,2,3], l=1)
│ ├── i=1 → swap(1,1) → [1,2,3]
│ │ └── fun([1,2,3], l=2) → print 123 → YES
│ └── i=2 → swap(1,2) → [1,3,2]
│ └── fun([1,3,2], l=2) → print 132 → NO
├── i=1 → swap(0,1) → [2,1,3]
│ └── fun([2,1,3], l=1)
│ ├── i=1 → swap(1,1) → [2,1,3]
│ │ └── fun([2,1,3], l=2) → print 213 → YES
│ └── i=2 → swap(1,2) → [2,3,1]
│ └── fun([2,3,1], l=2) → print 231 → NO
├── i=2 → swap(0,2) → [3,2,1]
│ └── fun([3,2,1], l=1)
│ ├── i=1 → swap(1,1) → [3,2,1]
│ │ └── fun([3,2,1], l=2) → print 321 → NO
│ └── i=2 → swap(1,2) → [3,1,2]
│ └── fun([3,1,2], l=2) → print 312 → NO
```

Final output: 123 213

(b) Generates all permutations of the array using backtracking and prints those satisfying a condition that  $\text{arr}[0] + \text{arr}[1] == 3$

**Q6.** Consider the Merge Sort algorithm.

- (a) Fill in the blanks in the `mergeSort` function: (2 marks)

```
void mergeSort(int arr[], int l, int r) {
 if (l < r) {
 int m = (l + r) / 2;

 -----;
 -----;
 merge(arr, l, m, r);
 }
}
```

- (b) Show complete steps of Merge Sort for the array: (2 marks)

[8, 3, 5, 2, 7]

- (c) What is the time complexity of Merge Sort in the worst case? Justify briefly. (1 mark)

Answer: (a) `mergeSort(arr, l, m); mergeSort(arr, m + 1, r);`

(b) [8,3,5,2,7]  
 → [8,3,5] [2,7]  
 → [8,3] [5] [2,7]  
 → [8] [3] [5] [2] [7]

Merge: [8],[3] → [3,8]  
 [3,8], [5] → [3,5,8]  
 [2],[7] → [2,7]  
 Final: [3,5,8] and [2,7] → [2,3,5,7,8]

They may also write:

```
mergeSort(arr, l, m-1); mergeSort(arr, m, r);
```

In that case, the split will be:

|                                   |                                                    |
|-----------------------------------|----------------------------------------------------|
| (b) [8,3,5,2,7]                   | Merge: [8],[3] $\rightarrow$ [3,8]                 |
| $\rightarrow$ [8,3] [5,2,7]       | [2],[7] $\rightarrow$ [2,7]                        |
| $\rightarrow$ [8,3] [5] [2,7]     | [5],[2,7] $\rightarrow$ [2,5,7]                    |
| $\rightarrow$ [8] [3] [5] [2] [7] | Final: [3,8] and [2,5,7] $\rightarrow$ [2,3,5,7,8] |

(c)  $O(n \log n)$  Reason:  $\log n$  levels, each level does  $O(n)$  merging